# Chapter 4: SQL Language

# Content

1. Introduction
2. Types of database languages
3. Data Definition Language - DDL
4. Data Manipulation Language - DML
5. Structured Query Language - SQL
6. Data Control Language - DCL

# 1. Introduction

- The origin of SQL is SEQUEL - *Structured English Query Language,* 1974 *)*
- *SQL standards:*
  - SQL89
  - SQL92 (SQL2)
  - SQL99 (SQL3)

→ The standard language for querying and manipulating relational databases.

# 2. Types of database languages

- **Data Definition Language** ( *DDL* ): allows declaring table structure, relationships and constraints.
- **Data Manipulation Language** ( *DML* ): allows adding, deleting, and editing data.
- **Structured Query Language** ( SQL ): allows querying data.
- **Data Control Language** ( *DCL* ): declares information security, grant and revoke access rights.

# 3. Data Definition Language

**3.1 CREATE command to create table**

3.1.1 Syntax

3.1.2 Some data types

**3.2 ALTER command to modify table structure**

3.2.1 Add attributes

3.2.2 Modify the data type of an attribute

3.2.3 Delete attributes

3.2.4 Add integrity constraints

3.2.5 Delete integrity constraints

**3.3 DROP command to delete table**

# 3.1 CREATE TABLE

**3.1.1 Syntax**

```
CREATE TABLE <table_name>
(
    <column_name1> <data_type> [not null],
    <column_name2> <data_type> [not null],
    …
    <column_name1> <data_type> [not null],
    <declare primary key, foreign key, constraint>
);
```

# 3.1 CREATE TABLE (2)

## 3.1.2 Some data types

| Data type | SQL Server |
|---|---|
| String | varchar(n), char(n), nvarchar(n), nchar(n) |
| Numeric | tinyint, smallint, int,<br>numeric(m,n), decimal(m,n), float, real,<br>smallmoney, money |
| Date | smalldatetime, datetime |
| Logic | bit |

# 3.1 CREATE TABLE (3)

***The sales management database schema includes the following relations:***

**KHACHHANG** (MAKH, HOTEN, DCHI, SODT, NGSINH, DOANHSO, NGDK, CMND)

**NHANVIEN** (MANV,HOTEN, NGVL, SODT)

**SANPHAM** (MASP,TENSP, DVT, NUOCSX, GIA)

**HOADON** (SOHD, NGHD, MAKH, MANV, TRIGIA)

**CTHD** (SOHD,MASP,SL)

# 3.1 CREATE TABLE (4)

```
CREATE TABLE KHACHHANG
(
    MAKH        char(4) PRIMARY KEY,
    HOTEN       varchar(40),
    DCHI        varchar(50),
    SODT        varchar(20),
    NGSINH      smalldatetime,
    DOANHSO     money,
    NGDK        smalldatetime,
    CMND        varchar(10)
);
```

# 3.1 CREATE TABLE (5)

```
CREATE TABLE CTHD
(
    SOHD    int FOREIGN KEY REFERENCES HOADON(SOHD),
    MASP    char(4) FOREIGN KEY REFERENCES SANPHAM(MASP),
    SL      int,
    CONSTRAINT PK_CTHD  PRIMARY KEY (SOHD,MASP)
);
```

# 3.2 ALTER TABLE (1)

## 3.2.1 Add attributes

***ALTER TABLE table name ADD column name data type***

Example: add column ghi_chu to the customer table

***ALTER TABLE*** *KHACHHANG* ***ADD*** *GHI_CHU varchar(20);*

## 3.2.2 Edit attribute data type

***ALTER TABLE table name ALTER COLUMN column name new_data_type***

Example: Change the Ghi_chu column to data type varchar(50)

***ALTER TABLE*** *KHACHHANG* ***ALTER COLUMN*** *GHI_CHU varchar(50)*

- **Note:** new data type must be compatible

# 3.2 ALTER TABLE (2)

- **New data type must be compatible, example:**
  - Cannot change from string type to number type
  - Cannot change from varchar(50) to varchar(5), if the column already contains values with a length of more than 5 characters.

## 3.2.3 Delete attribute

***ALTER TABLE table_name DROP COLUMN column_name***

Example: delete the ghi_chu column in the customer table.

***ALTER TABLE*** *KHACHHANG* ***DROP COLUMN*** *ghi_chu*

# 3.2 ALTER TABLE (3)

## 3.2.4 Add integrity constraints

**ALTER TABLE**
<table_name>
**ADD CONSTRAINT**
<constraint_name>

- **UNIQUE column_name**
- **PRIMARY KEY (column_name)**
- **FOREIGN KEY (column_name) REFERENCES table_name (primary_key_column) [ON UPDATE CASCADE] [ON DELETE CASCADE | SET NULL | SET DEFAULT]**
- **CHECK (column_name condition)**

# 3.2 ALTER TABLE (4)

- **Example:**
  - **ALTER TABLE** KHACHHANG **ADD CONSTRAINT** UQ_KH **UNIQUE** (CCCD)
  - **ALTER TABLE** NHANVIEN **ADD CONSTRAINT** PK_NV **PRIMARY KEY** (MANV)
  - **ALTER TABLE** CTHD **ADD CONSTRAINT** FK_CT_SP **FOREIGN KEY** (MASP) **REFERENCES** SANPHAM(MASP)
  - **ALTER TABLE** SANPHAM **ADD CONSTRAINT** CK_GIA **CHECK** (GIA >=500)

# 3.2 ALTER TABLE (5)

## 3.2.5 Delete integrity constraints

***ALTER TABLE table_name DROP CONSTRAINT constraint_name***

**Example**:

- ALTER TABLE CTHD DROP CONSTRAINT FK_CT_SP
- ALTER TABLE SANPHAM DROP CONSTRAINT ck_gia

- **Note**: To delete primary key constraints, you must first delete all foreign key constraints that reference it.

# 3.3 DROP TABLE

- **Syntax:** DROP TABLE table_name

  Example: delete both the structure and content of the customer table.

  DROP TABLE KHACHHANG

- **Note**: When you want to drop a table, you must first delete all foreign keys that reference that table.

# 4. Data Manipulation Language

4.1 INSERT command to add data

4.2 UPDATE command to edit data

4.3 DELETE command to delete data

# 4.1 INSERT data

- **Syntax**

  INSERT INTO table_name VALUES (value_1, value_2,…, value_n)

- **Advanced syntax**

  INSERT INTO table_name(column1,…,columnn) VALUES (value_1,…., value_n)

  → *we can specify some columns only and their order.*

# 4.1 INSERT data

- **Example:**
  - INSERT INTO SANPHAM VALUES ('BC01', 'But chi', 'cay', 'Singapore', 3000)
  - INSERT INTO SANPHAM(masp,tensp,dvt,nuocsx,gia) VALUES ('BC01', 'But chi', 'cay', 'Singapore', 3000)

# 4.2 UPDATE data

- **Syntax**

  UPDATE table_name
  SET column_1 = value_1, column_2 = value_2 ….
  [WHERE condition]

- **Advanced syntax**

  UPDATE ta
  SET ta.column_1 = value_1, ta.column_2 = value_2 ….
  FROM TableA AS ta JOIN TableB AS tb ON <condition>
  [WHERE condition]

# 4.2 UPDATE data

- **Example**: Increase price by 10% for all Chinese products

```
UPDATE PRODUCT
SET Price = Price*1.1
WHERE Country='China'
```

- **Note**:
  - SQL operations work **row-by-row**.

# 4.3 DELETE data

- **Syntax**

  DELETE FROM table_name [WHERE condition]

- **Example:** Remove Chinese products priced below 10000

  DELETE FROM SANPHAM
  WHERE (Nuocsx='China') and (Price<10000)

- **Note**:
  - Be careful, update or delete commands WITHOUT WHERE condition will update or delete all rows.

# 5. Structured Query Language

5.1 General query syntax
5.2 Simple query
5.3 Multi-table query, join
5.4 Aliasing, using *, distinct
5.5 Operators
5.6 Subquery
5.7 Division
5.8 Aggregation functions and grouping

# 5.1 General query syntax

```
SELECT [DISTINCT] *|column|function
FROM table
[WHERE condition]
[GROUP BY column_name]
[HAVING condition]
[ORDER BY column_name ASC|DESC]
```

# 5.2 Simple query (1)

SELECT

- Corresponding to the **projection** of relational algebra
- List the attributes to display in the results

FROM

- List the input relations, their joins.

WHERE

- Corresponding to the **selection** of relational algebra
- Conditions related to attributes, filtering on the rows
- Using logical operators AND, OR, NOT, comparison operators, BETWEEN … AND …

# 5.2 Simple query (2)

- Find masp, tensp made in China with price from 20000 to 30000

```
SELECT masp, tensp
FROM SANPHAM
WHERE  nuocsx='CHINA'
    AND gia BETWEEN 20000 AND 30000
```

# 5.3 JOIN (1)

- Table1 [**INNER] JOIN** Table2 **ON <condition>**
- Table1 **LEFT [OUTER] JOIN** Table2 **ON <condition>**
- Table1 **RIGHT [OUTER] JOIN** Table2 **ON <condition>**
- Table1 **FULL [OUTER] JOIN** Table2 **ON <condition>**

# 5.3 JOIN ( 2 )

- **Example:** Print out the list of customers (MAKH, HOTEN) who made purchases on January 1, 2007.

(Explicit join)

- SELECT KHACHHANG.makh, hoten
  FROM KHACHHANG **JOIN** HOADON ON
  KHACHHANG.makh=HOADON.makh
  WHERE nghd='1/1/2007'

(Implicit join, should **not** use)

- SELECT KHACHHANG.makh, hoten
  FROM KHACHHANG, HOADON
  WHERE KHACHHANG.makh=HOADON.makh AND
  nghd='1/1/2007'

# 5.3 JOIN ( 3 )

- **Example:** Print out a list of all invoices and the name of the customer who purchased that invoice (if any)
  - SELECT sohd, hoten
    FROM HOADON **LEFT JOIN** KHACHHANG ON HOADON.makh=KHACHHANG.makh
- **Example:** Print out a list of all invoices and the names of all customers with their purchased invoices (if any)
  - SELECT sohd, hoten
    FROM HOADON **FULL JOIN** KHACHHANG ON HOADON.makh=KHACHHANG.makh

# 5.4 Aliasing, using *, distinct

- Alias for attributes and relationships:

  `old_name AS new_name`

  - SELECT manv, hoten AS [fullname] FROM NHANVIEN
- List all the attributes of the relation:
  - SELECT * FROM NHANVIEN
  - SELECT NHANVIEN.* FROM NHANVIEN, HOADON
- Distinct: de-duplicate results
  - SELECT DISTINCT nuocsx FROM SANPHAM
- Sort the results: ORDER BY
  - SELECT * FROM SANPHAM
    ORDER BY nuocsx ASC, gia DESC

# 5.5 Query Operators (1)

- Comparison operators: =, >,<,>=,<=,<>
- Logical operators: AND, OR, NOT
- Math operations: +, - ,* , /
- x BETWEEN a AND b ($\Leftrightarrow a \leq x \leq b$)
- IS NULL, IS NOT NULL
- LIKE (_%)
- IN, NOT IN
- EXISTS, NOT EXISTS
- SOME, ALL

# 5.5 Query Operators (2)

- **IS NULL, IS NOT NULL**
  - SELECT sohd FROM HOADON WHERE makh IS NULL
  - SELECT * FROM HOADON WHERE makh IS NOT NULL
- **Comparison operators and operations**
  - SELECT price*1.1 AS [sale price] FROM SANPHAM WHERE nuocsx<>'Viet Nam'
  - SELECT * FROM SANPHAM WHERE (price BETWEEN 20000 AND 30000) OR (country='Vietnam')
- **IN, NOT IN Operators**
  - SELECT * FROM SANPHAM WHERE masp NOT IN ('BB01', 'BB02', 'BB03')

# 5.5 Comparison operators (3)

**LIKE operator**

- Relative string comparison (simplified regex)
- Syntax: s LIKE p, p can contain % or _
  - ***% : replace any string of characters***
  - ***_ : replace any 1 character***
  - ***[0-9] : 1 arbitrary digit***
  - ***[a-z] : 1 arbitrary lowercase letter***
  - ***[A-Z] : 1 arbitrary uppercase letter***
  - ***^ : negative, not containing the pattern***
- **Example:** SELECT masp, tensp FROM SANPHAM WHERE masp LIKE 'B%01'

# 5.6 Subquery (1)

## IN or EXISTS

- **Example:** Find the invoices for purchasing 2 products with codes "BB01" and "BB02" at the same time.
  - SELECT DISTINCT sohd FROM CTHD
    WHERE masp='BB01'
    AND **sohd IN** ( SELECT DISCTINCT sohd FROM CTHD WHERE masp='BB02' )
  - SELECT DISTINCT A.sohd FROM CTHD A
    WHERE A.masp='BB01'
    AND **EXISTS** ( SELECT * FROM CTHD B WHERE B.masp='BB02' AND **A.sohd=B.sohd** )

# 5.6 Subquery (2)

## NOT IN or NOT EXISTS

- **Example:** Find invoices that purchased product code 'BB01' but did not purchase product code 'BB02'.
  - SELECT DISTINCT sohd FROM CTHD
    WHERE masp='BB01'
    AND **sohd NOT IN** ( SELECT DISTINCT sohd FROM CTHD WHERE masp='BB02')
  - SELECT DISTINCT A.sohd FROM CTHD A
    WHERE A.masp='BB01'
    AND **NOT EXISTS** ( SELECT * FROM CTHD B WHERE B.masp='BB02' AND **A.sohd=B.sohd**)

# IN vs. EXISTS

- IN
  - Check whether a value in the parent query is in the results of a subquery.
  - Must list the attribute to check, and their data types must be compatible.
- EXISTS
  - No other attribute, constants, or expressions in the WHERE clause need to come before the subquery.
  - It is not necessary to list the attribute names in the SELECT clause of the subquery.
  - Queries with =ANY or IN can be converted to queries with EXISTS.

# EXIST and NOT EXIST

- **EXIST** and **NOT EXIST only return True** or **False** values.
- Value of **EXIST:**
  - **True:** when the subquery returns **any** tuples
  - **False:** when the subquery returns **no** tuples
- Value of **NOT EXIST:**
  - **True:** when the subquery returns **no** tuples
  - **False:** when the subquery returns **any** tuples

# 5.7 Division

**Use NOT EXISTS**

- **Example:** Find Pilots who can fly all planes.

```
SELECT MAPC FROM PHICONG
WHERE NOT EXISTS (1)
 ( SELECT * FROM PHICO WHERE NOT EXISTS (2)
     ( SELECT * FROM CHUNGNHAN CN
        WHERE PHICO.MAPHICO=CN.MAPHICO
        AND CN.MAPC=PHICONG.MAPC ))
```

# 5.7 Division (cont.)

| PHICONG | |
|---|---|
| MAPC | TENPC |
| A | Peter |
| B | John |
| C | Red |

| PHICO | |
|---|---|
| MAPHICO | TENPHICO |
| X | Boeing-777 |
| Y | Boeing-787 |
| Z | Airbus-A380 |

| CHUNGNHAN | |
|---|---|
| MAPC | MAPHICO |
| A | X |
| A | Y |
| B | Z |
| C | X |
| C | Y |
| C | Z |

[A,X] -> (2) wrong -> do not choose
[A,Y] -> (2) wrong -> do not choose
[A,Z] -> (2) correct -> choose
=> (1) is wrong bc of Z => not choose A

[B,X] -> (2) correct -> choose
[B,Y]-> (2) correct -> choose
[B,Z]> (2) wrong -> do not choose
=> (1) is wrong bc of X,Y => not choose B

[C,X] -> (2) Wrong -> do not choose
[C,Y] -> (2) wrong -> do not choose
[C,Z] -> (2) wrong -> do not choose
=> (1) is correct bc there is none => choose C

⇒ C is a pilot who can fly all planes.

# 5.7 Division (cont.)

- **Example:** Find the invoice numbers that have purchased all products made in “Singapore”.

```
SELECT sohd FROM HOADON
WHERE NOT EXISTS
    ( SELECT * FROM SANPHAM
    WHERE nuocsx='Singapore'
        AND NOT EXISTS
            ( SELECT * FROM CTHD
            WHERE HOADON.sohd=CTHD.sohd AND
            CTHD.masp=SANPHAM.masp ))
```

# 5.7 Division: **Intuition**

- **Example:** Find the pilots who can fly all planes.
  → Intuition: Find pilots, **such that there is no** planes, that the pilot **cannot fly**.

- **Example:** Find the invoices that bought all products manufactured in "Singapore".
  → Intuition: Find invoices, **such that there is no** products manufactured in "Singapore", that the invoice **did not purchase**.

**⇒ A÷B = Find A, such that there is no B, that cannot join A-X-Y-Z-...-B.**

# 5.8 Aggregation functions and grouping (1)

## 5.8.1 Basic calculation functions

- COUNT: Counts the number of values of an attribute
- MIN: Calculate the smallest value
- MAX: Calculates the largest value
- AVG: Calculate the average value
- SUM: Calculates the total value of data sets

| NHANVIEN | | | | | |
|---|---|---|---|---|---|
| **MANV** | **HOTEN** | **PHAI** | **MANQL** | **PHONG** | **LUONG** |
| NV001 | Nguyễn Ngọc Linh | Nữ | Null | NC | 2.800.000 |
| NV002 | Đinh Bá Tiến | Nam | NV002 | DH | 2.000.000 |
| NV003 | Nguyễn Văn Mạnh | Nam | NV001 | NC | 2.300.000 |
| NV004 | Trần Thanh Long | Nam | NV002 | DH | 1.800.000 |
| NV005 | Nguyễn Thị Hồng Vân | Nữ | NV001 | NC | 2.500.000 |
| NV006 | Nguyễn Minh | Nam | NV002 | DH | 2.000.000 |
| NV007 | Hà Duy Lập | Nam | NV003 | NC | 1.800.000 |
| NV008 | Trần Kim Duyên | Nữ | NV003 | NC | 1.800.000 |
| NV009 | Nguyễn Kim Anh | Nữ | NV003 | NC | 2.000.000 |

# For example

1. Calculate the lowest, highest, average and total salary of all employees.
2. How many employees are there in total?
3. How many employees have a manager?
4. How many departments have direct employees?
5. Calculate the average salary of employees
6. Calculate the average salary of employees for each department

1. Calculate the lowest, highest, average and total salary of all employees.

```
SELECT   MIN(luong) AS thapnhat,
         MAX(luong) AS caonhat,
         AVG(luong) AS trungbinh,
         SUM(luong) AS tongluong
FROM  NhanVien
```

2. How many employees are there in total?

SELECT **COUNT(*)** FROM NhanVien

3. How many employees have a manager?

- SELECT COUNT(*) FROM NhanVien

  WHERE manql IS NOT NULL
- SELECT **COUNT(manql)** FROM NhanVien

4. How many departments have direct employees?

SELECT **COUNT(DISTINCT phong)** FROM NhanVien

# 5.8 Aggregation functions and grouping (2)

## 5.8.2 GROUP BY clause

- The GROUP BY clause specifies grouping attributes. Those attributes must appear in the SELECT clause.
- Each group consists of a set of rows with the same values on the grouping attributes.

5. Calculate the average salary of employees

```
SELECT    AVG(luong) AS LUONGTB
FROM      NhanVien
```

6. Calculate the average salary of employees for each department

```
SELECT      phong, AVG(luong) AS LUONGTB
FROM        NhanVien
GROUP BY    phong
```

# 5.8 Aggregation functions and grouping (3)

## 5.8.3 Post-grouping condition: HAVING clause

- Filter results based on aggregation functions after grouping.
- Unlike the WHERE clause, the HAVING clause filters on groups rather than on individual rows.

- **Example:** Find offices with more than 5 “Female” employees.

```
SELECT      phong
FROM        NhanVien
WHERE       phai = ‘Nữ’
GROUP BY    phong
HAVING      COUNT(manv) > 5
```

# BÀI TẬP

***Lược đồ CSDL quản lý bán hàng gồm có các quan hệ sau:***

**KHACHHANG** (MAKH, HOTEN, DCHI, SODT, NGSINH, DOANHSO, NGDK)

**NHANVIEN** (MANV, HOTEN, NGVL, SODT)

**SANPHAM** (MASP, TENSP, DVT, NUOCSX, GIA)

**HOADON** (SOHD, NGHD, MAKH, MANV, TRIGIA)

**CTHD** (SOHD, MASP, SL)

# Câu hỏi

Write SQL script to do the following tasks:

1. Declare table CTHD.
2. Add columns to table SANPHAM: Description, ReviewSummary, Rating.
3. Increase price by 10% for all product items that have been purchased this year.
4. Find MaNV, HoTen that have sold some products made in VN.
5. Find MaKH, HoTen that have bought all products priced more than 1 million.